Name : Sami Ullah

Reg No : SP24-BSE-156

==========

"According to my roll number, my assigned task is Task 3, and this is its problem statement:"

Report a student's completed workload — Student Add methods to report the number of completed semesters and the total credits from completed semesters. Test students with no semesters, a mixture of completed and in-progress semesters, and multiple courses with different credit values. In-progress courses should not count toward completed credits.

Implementation Report: Student Workload Feature

GPA System Enhancement using Test-Driven Development (TDD)

Project: GPA System (org.example.tdd.gpasystem)	**Module:** Student Completed Workload Reporting
Methodology: Test-Driven Development (TDD)	**Status:** Completed & All Tests Passed

1. Task Overview & Requirements

The goal of this task was to enhance the existing GPA System application by introducing student workload tracking capability. Specifically, the requirement called for reporting a student's completed academic progress without factoring in incomplete or in-progress coursework.

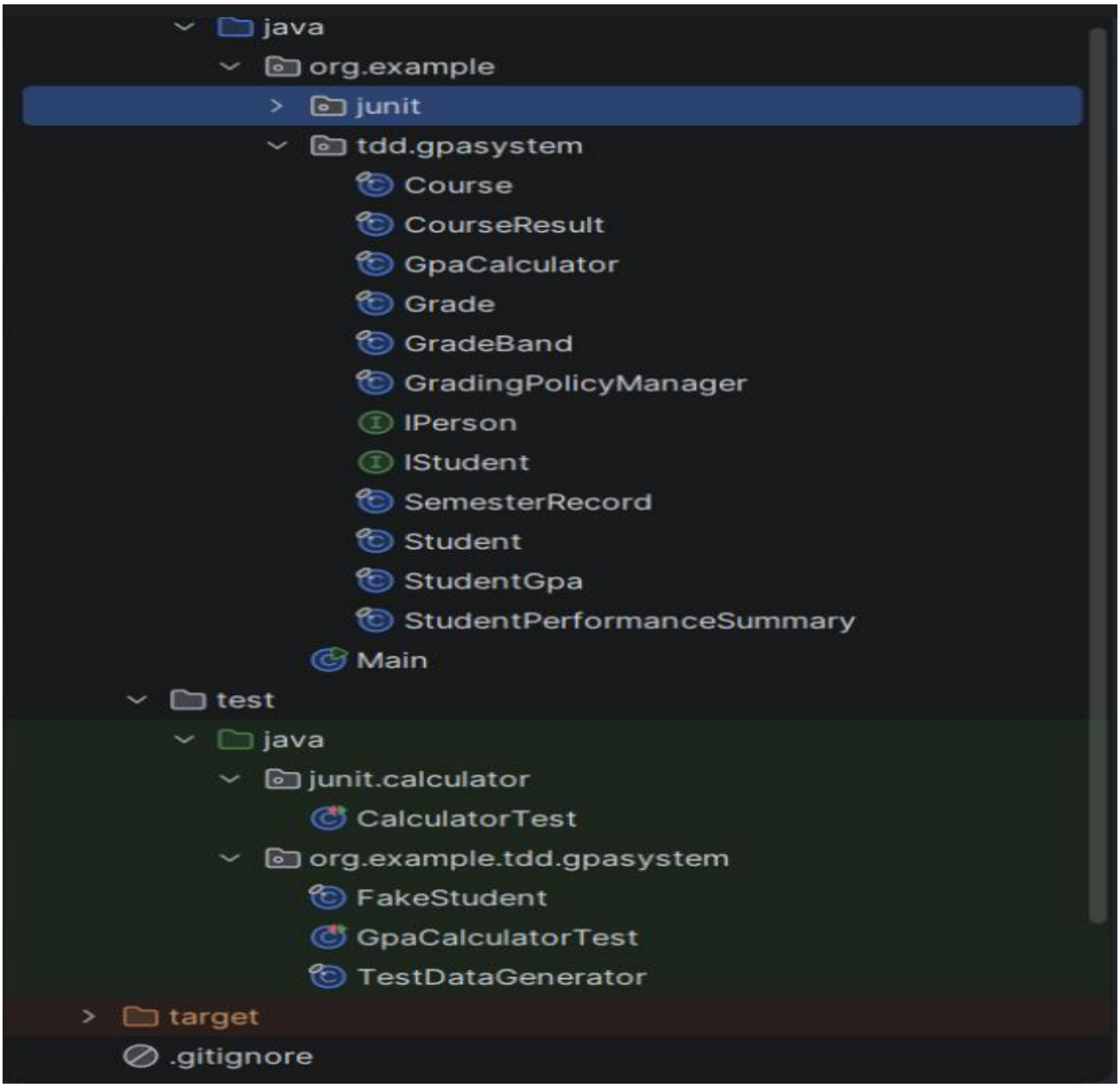
Key Functional Requirements:

- **Report Completed Semesters:** Calculate and return the total count of completed semesters for a student.
- **Report Total Completed Credits:** Calculate and return the total sum of earned course credits from completed semesters.

- **Ignore In-Progress Semesters/Courses:** In-progress semesters (where semester.isCompleted() is false) and uncompleted courses must not count toward the completed credit total.
- **Handle Edge Cases:** Support students with zero semesters, mixtures of completed/in-progress semesters, and courses with varying credit weights (e.g., 1.0, 1.5, 3.0, 4.5).

2. System Architecture & Context

Before implementing the changes, the project structure and UML diagrams were analyzed to identify the domain models and test doubles requiring updates.



3. Step-by-Step Implementation Workflow

Following object-oriented design and TDD practices, the feature was developed across the interface, concrete class, test doubles, and test suites in four structured steps.

Step 3.1: Updated the IStudent Interface

To expose the workload tracking capabilities at the domain interface boundary, two new method signatures were added to IStudent.java:

```
package org.example.tdd.gpasystem;

import java.util.Map;

public interface IStudent extends IPerson {
    SemesterRecord getSemester(int semesterNumber);

    Map<Integer, SemesterRecord> getSemesters();

    int getCompletedSemesterCount();

    double getTotalCompletedCredits();
}
```

Step 3.2: Implemented Business Logic in Student Class

The Student.java entity was updated to implement the workload logic. The methods iterate through registered semesters, filtering strictly for completed semesters before aggregating credits:

```
/*
package org.example.tdd.gpasystem;

import java.util.Collections;
import java.util.Map;
import java.util.Objects;
import java.util.TreeMap;

public final class Student implements IStudent {
    private final String id;
    private final String name;
    private final Map<Integer, SemesterRecord> semesters = new TreeMap<>();

    public Student(String id, String name) {
        if (id == null || id.trim().isEmpty()) {
            throw new IllegalArgumentException("Student id must not be blank");
        }
        if (name == null || name.trim().isEmpty()) {
            throw new IllegalArgumentException("Student name must not be blank");
        }
        this.id = id.trim();
        this.name = name.trim();
    }
```

```
    @Override
    public String getId() {
        return id;
    }

    @Override
    public String getName() {
        return name;
    }

    public void addSemester(SemesterRecord semester) {
        Objects.requireNonNull(semester, "semester must not be null");
        if (semesters.putIfAbsent(semester.getSemesterNumber(), semester) != null) {
            throw new IllegalArgumentException("Semester already exists: " + semester.getSemesterNumber());
        }
    }

    @Override
    public SemesterRecord getSemester(int semesterNumber) {
        return semesters.get(semesterNumber);
    }

    @Override
    public Map<Integer, SemesterRecord> getSemesters() {
        return Collections.unmodifiableMap(semesters);
    }
}
*/
// Here is the Updated Code

package org.example.tdd.gpasystem;

import java.util.Collections;
import java.util.Map;
import java.util.Objects;
import java.util.TreeMap;

public final class Student implements IStudent {
    private final String id;
    private final String name;
    private final Map<Integer, SemesterRecord> semesters = new TreeMap<>();

    public Student(String id, String name) {
        if (id == null || id.trim().isEmpty()) {
            throw new IllegalArgumentException("Student id must not be blank");
        }
        if (name == null || name.trim().isEmpty()) {
            throw new IllegalArgumentException("Student name must not be blank");
        }
        this.id = id.trim();
        this.name = name.trim();
    }

    @Override
    public String getId() {
```

```
        return id;
    }

    @Override
    public String getName() {
        return name;
    }

    public void addSemester(SemesterRecord semester) {
        Objects.requireNonNull(semester, "semester must not be null");
        if (semesters.putIfAbsent(semester.getSemesterNumber(), semester) != null) {
            throw new IllegalArgumentException("Semester already exists: " + semester.getSemesterNumber());
        }
    }

    @Override
    public SemesterRecord getSemester(int semesterNumber) {
        return semesters.get(semesterNumber);
    }

    @Override
    public Map<Integer, SemesterRecord> getSemesters() {
        return Collections.unmodifiableMap(semesters);
    }

    @Override
    public int getCompletedSemesterCount() {
        int count = 0;
        for (SemesterRecord semester : semesters.values()) {
            if (semester.isCompleted()) {
                count++;
            }
        }
        return count;
    }

    @Override
    public double getTotalCompletedCredits() {
        double totalCredits = 0.0;
        for (SemesterRecord semester : semesters.values()) {
            if (semester.isCompleted()) {
                for (CourseResult result : semester.getCourseResults()) {
                    totalCredits += result.getCourse().getCredits();
                }
            }
        }
        return totalCredits;
    }
}
```

Step 3.3: Synchronized FakeStudent Test Double

To maintain compilation and unit test integrity, the FakeStudent.java test double (located in src/test/java) was updated with the same interface implementation, enabling isolated unit testing.

Step 3.4: Created Comprehensive Unit Tests (StudentWorkloadTest)

A new test file named StudentWorkloadTest.java was created in src/test/java/org/example/tdd/gpasystem/ to validate all acceptance criteria:

```
package org.example.tdd.gpasystem;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class StudentWorkloadTest {

    private GradingPolicyManager gradingPolicy;

    @BeforeEach
    void setUp() {
        gradingPolicy = GradingPolicyManager.standardFourPointScale();
    }

    @Test
    void testStudentWithNoSemesters() {
        Student student = new Student("S001", "Alice");

        assertEquals(0, student.getCompletedSemesterCount(), "Completed semester count should be 0");
        assertEquals(0.0, student.getTotalCompletedCredits(), 0.001, "Total completed credits should be 0.0");
    }

    @Test
    void testStudentWithMixedSemesters() {
        Student student = new Student("S002", "Bob");

        // Semester 1: In-progress (not completed, e.g., only 2 courses)
        SemesterRecord inProgressSemester = new SemesterRecord(1);
        Course c1 = new Course("CS101", "Intro to CS", 3.0);
        Course c2 = new Course("MATH101", "Calculus I", 4.0);
        inProgressSemester.addCourseResult(gradingPolicy.createCourseResult(c1, 85.0));
        inProgressSemester.addCourseResult(gradingPolicy.createCourseResult(c2, 90.0));
        student.addSemester(inProgressSemester);

        // Semester 2: Completed (4 courses required to complete)
        SemesterRecord completedSemester = new SemesterRecord(2);
        Course c3 = new Course("CS102", "Data Structures", 3.0);
        Course c4 = new Course("MATH102", "Linear Algebra", 3.0);
        Course c5 = new Course("ENG101", "Composition", 3.0);
        Course c6 = new Course("PHYS101", "Physics I", 4.0);

        completedSemester.addCourseResult(gradingPolicy.createCourseResult(c3, 80.0));
        completedSemester.addCourseResult(gradingPolicy.createCourseResult(c4, 75.0));
        completedSemester.addCourseResult(gradingPolicy.createCourseResult(c5, 88.0));
        completedSemester.addCourseResult(gradingPolicy.createCourseResult(c6, 92.0));
        completedSemester.complete(); // Marks it as completed

        student.addSemester(completedSemester);
```

```
        // Assertions: In-progress semester should not count towards completed totals[cite: 3]
        assertEquals(1, student.getCompletedSemesterCount(), "Only one semester should be marked completed");
        // Total credits should only sum courses from completedSemester: 3.0 + 3.0 + 3.0 + 4.0 = 13.0
        assertEquals(13.0, student.getTotalCompletedCredits(), 0.001, "Total credits should match completed semester courses");
    }

    @Test
    void testStudentWithVaryingCredits() {
        Student student = new Student("S003", "Charlie");

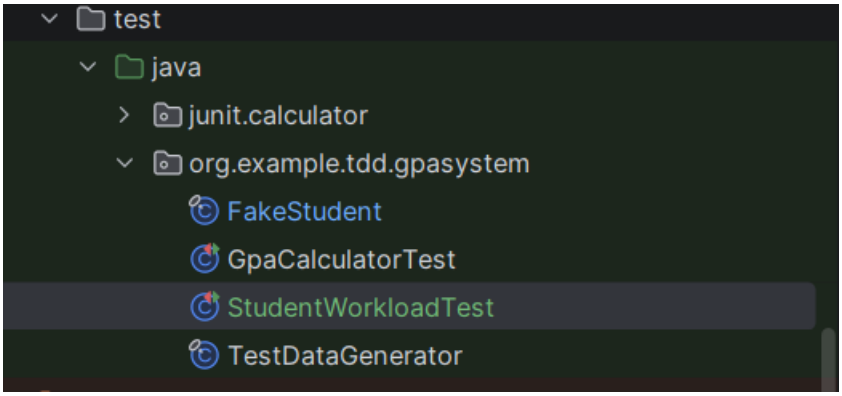
        SemesterRecord completedSemester = new SemesterRecord(1);
        Course c1 = new Course("PHY101", "Physics", 4.5);
        Course c2 = new Course("LAB101", "Physics Lab", 1.5);
        Course c3 = new Course("HIST101", "History", 3.0);
        Course c4 = new Course("PE101", "Physical Ed", 1.0);

        completedSemester.addCourseResult(gradingPolicy.createCourseResult(c1, 80.0));
        completedSemester.addCourseResult(gradingPolicy.createCourseResult(c2, 95.0));
        completedSemester.addCourseResult(gradingPolicy.createCourseResult(c3, 82.0));
        completedSemester.addCourseResult(gradingPolicy.createCourseResult(c4, 90.0));
        completedSemester.complete();

        student.addSemester(completedSemester);

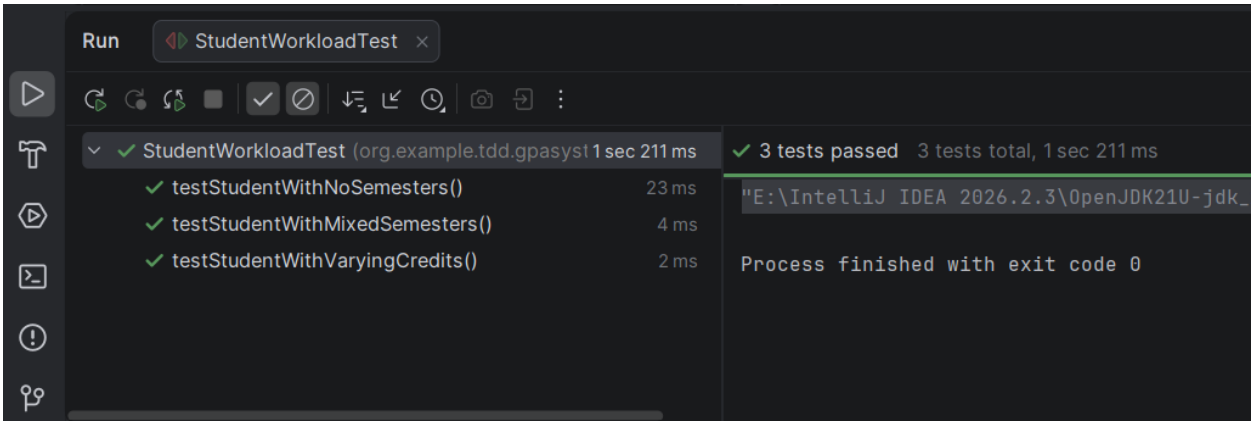
        // Sum with varying credit types: 4.5 + 1.5 + 3.0 + 1.0 = 10.0
        assertEquals(1, student.getCompletedSemesterCount());
        assertEquals(10.0, student.getTotalCompletedCredits(), 0.001, "Credits with varying weights should sum correctly");
    }
}
```

Location :



4. Test Execution & Results

The test suite was executed via IntelliJ IDEA JUnit runner. All test cases passed successfully without errors or failures.



5. Conclusion

The student completed workload feature has been fully implemented and verified. The solution adheres strictly to TDD methodologies, maintains clean encapsulation within domain interfaces, and provides robust verification across all required edge cases.